as all requests need to share the service. It must be thread-safe, though. Similarly, you might register a settings object loaded from a remote server as a singleton if you load the settings once at startup and reuse them through the lifetime of your app.

On the face of it, choosing a lifetime for a service may not seem to be too tricky. But an important gotcha can come back to bite you in subtle ways, as you'll see in section 9.4.4.

9.4.4 Keeping an eye out for captive dependencies

Suppose that you're configuring the lifetime for the `DataContext` and `Repository` examples. You think about the suggestions I've provided and decide on the following lifetimes:

- `DataContext`—Scoped, as it should be shared for a single request
- `Repository`—Singleton, as it has no state of its own and is thread-safe, so why not?

WARNING This lifetime configuration is to explore a bug. Don't use it in your code; if you do, you'll experience a similar problem!

Unfortunately, you've created a captive dependency because you're injecting a scoped object, `DataContext`, into a singleton, `Repository`. As it's a singleton, the same `Repository` instance is used throughout the lifetime of the app, so the `DataContext` that was injected into it will also hang around, even though a new one should be used with every request. Figure 9.10 shows this scenario, in which a new instance of